

PENETRATION TESTING DOCUMENT

By: KRYSTIAN BOCHENKO & ETHAN NAYAK

Summary

This penetration testing report describes the security assessment of the Secure Document Sharing System built for CS 419. The application is a Flask-based web app that lets users upload, share, and manage encrypted documents with role-based access control.

Testing was performed by the development team (Krystian Bochenko and Ethan Nayak) using both manual methods and automated tools, including OWASP ZAP and Burp Suite Community Edition. All major OWASP Top 10 vulnerability categories were reviewed.

No critical vulnerabilities were found. The application showed strong security protections across all tested areas. Minor informational findings are listed below, along with recommended fixes.

Severity	Count	Status
Critical	0	None
High	0	None
Medium	1	Documented
Low	2	Documented
Informational	3	Documented

Testing

Testing followed the OWASP Web Security Testing Guide (WSTG) methodology. All tests were run in a local development environment against the application at <https://127.0.0.1:5000>.

Tools Used

OWASP ZAP: Automated scanning for common web vulnerabilities like XSS, injection, and configuration issues

Burp Suite Community: Manual interception and modification of HTTP/HTTPS requests

Browser DevTools: Checking security headers and analyzing cookies

Manual Testing: Direct URL changes, form tampering, and attempts to escalate privileges

Python Scripts: Custom scripts for brute-force and rate-limit testing

Scope

Authentication system (registration, login, logout)

Access control and role-based permissions

Input validation and protection against injection

Session management

File upload and download features

Security header setup

Transport layer security (HTTPS/TLS)

Encryption of stored data

Testing Approach

Testing was done in three phases: (1) automated scanning with OWASP ZAP to find basic issues, (2) manual testing of logic and access control, and (3) checking security measures against the OWASP Top 10 checklist. All findings were recorded with steps to reproduce and suggested fixes.

```
(.venv) PS C:\Users\kryst\Desktop\securewebapp> powershell
>> pytest tests/script.py -v
>> ...
===== test session starts =====
platform win32 -- Python 3.13.13, pytest-9.0.3, pluggy-1.6.0 -- C:\Users\kryst\Desktop\securewebapp\.venv\Scripts\python.exe
cachedir: .pytest_cache
rootdir: C:\Users\kryst\Desktop\securewebapp
collected 11 items

tests/script.py::test_register_weak_password PASSED [ 9%]
tests/script.py::test_register_invalid_username PASSED [ 18%]
tests/script.py::test_register_mismatched_passwords PASSED [ 27%]
tests/script.py::test_login_invalid_credentials PASSED [ 36%]
tests/script.py::test_dashboard_requires_auth PASSED [ 45%]
tests/script.py::test_admin_requires_auth PASSED [ 54%]
tests/script.py::test_documents_requires_auth PASSED [ 63%]
tests/script.py::test_security_headers PASSED [ 72%]
tests/script.py::test_xss_in_username PASSED [ 81%]
tests/script.py::test_password_no_uppercase PASSED [ 90%]
tests/script.py::test_password_no_special_char PASSED [100%]

===== warnings summary =====
tests/script.py::test_register_weak_password
tests/script.py::test_register_invalid_username
tests/script.py::test_login_invalid_credentials
tests/script.py::test_xss_in_username
tests/script.py::test_password_no_uppercase
tests/script.py::test_password_no_special_char
  C:\Users\kryst\Desktop\securewebapp\security_logger.py:22: DeprecationWarning: datetime.datetime.utcnow() is deprecated and scheduled for removal in a future version. Use timezone-aware objects to represent datetimes in UTC:
  datetime.datetime.now(datetime.UTC).
    "timestamp": datetime.utcnow().isoformat(),

-- Docs: https://docs.pytest.org/en/stable/how-to/capture-warnings.html
===== 11 passed, 6 warnings in 0.22s =====
```

Test Categories & Checklists

Authentication Testing

Test	Result	Notes
Brute Force Protection	PASS	Lockout after 5 attempts
Password Complexity	PASS	12+ chars, complexity required
Session Management	PASS	Secure token, 30 min timeout
Logout Functionality	PASS	Session destroyed on logout
Password Reset	N/A	Feature not implemented
Rate Limiting (10/min per IP)	PASS	Flask-limiter implemented
Account Lockout (15 min)	PASS	Verified with wrong passwords

Authorization Testing

Test	Result	Notes
Horizontal Privilege Escalation	PASS	Users cannot access others docs
Vertical Privilege Escalation	PASS	Users cannot access admin pages
Direct Object Reference	PASS	Doc ID checked against ownership
Forced Browsing	PASS	All routes require authentication

Input Validation Testing

Test	Result	Notes
XSS Attempts In Username Field	PASS	Whitelist + html.escape blocks XSS
XSS Attempts In Login Field	PASS	No script execution observed
Path Traversal In Download URL	PASS	Returns 404, no file access
Command Injection In Filename	PASS	secure_filename() prevents injection
File Upload - Invalid Extension	PASS	Rejected with error message
File Upload - Invalid MIME Type	PASS	MIME type checked server-side
Empty Input Submission	PASS	Validation catches empty fields
Oversized Input (Email > 254)	PASS	Length limits enforced

Session Security Testing

Test	Result	Notes
Session Fixation	PASS	New token generated on each login
Session Hijacking	PASS	HttpOnly + Secure + SameSite = Strict
Concurrent Session Handling	INFO	Multiple Sessions Allowed
Session Timeout (30 min)	PASS	Verified By Reducing Timeout

Configuration Testing

Test	Result	Notes
Security Headers Present	PASS	All of the header (7) were verified using developer tools
TLS/SSL Configuration	PASS	Https along with a self-signed certificate
Error Handling	PASS	Generic messages shown to users
Debug mode in production	INFO	debug=True

Findings

MEDIUM-01: Debug Mode Enabled in Development

Severity: **Medium** Category: MEDIUM

Description: The Flask app runs with debug=True, which turns on the interactive debugger and shows full error details. This is fine for development but should not be used in production.

Impact: If left enabled in production, attackers could access the Werkzeug debugger, run arbitrary code, and see sensitive stack trace information.

Steps to Reproduce:

Start the app with python app.py

Trigger an error by visiting a broken page

See that the Werkzeug debugger appears with full stack trace

Proof of Concept: Werkzeug debugger is visible on error pages during testing.

Remediation: Set debug=False and use environment variables (e.g., app.run(debug=os.environ.get('FLASK_DEBUG', False))). Never deploy with debug mode enabled.

LOW-01: Self-Signed TLS Certificate

Severity: **Low** Category: LOW

Description: The app uses a self-signed TLS certificate created with pyOpenSSL. Browsers warn users that the connection is not trusted.

Impact: Users may get used to ignoring certificate warnings, which makes it harder to spot real man-in-the-middle attacks. Risk is low in a development or academic setting.

Steps to Reproduce:

Go to <https://127.0.0.1:5000>

See the browser warning about an untrusted certificate

Check that the certificate is self-signed

Proof of Concept: Browser shows a “Your connection is not private” warning on all pages.

Remediation: For production, use a certificate from a trusted Certificate Authority (e.g., Let’s Encrypt). Self-signed certificates are fine for development.

LOW-02: No Password Reset Functionality

Severity: **Low** Category: LOW

Description: The app does not have a password reset feature. Users cannot recover their account without admin help.

Impact: Low security risk, but it affects usability. Admins must reset passwords manually, which does not scale well.

Steps to Reproduce:

Go to the login page

See there is no “Forgot Password” option

Confirm there is no password reset route in the app

Proof of Concept: No reset link on the login page and no /reset-password endpoint exists.

Remediation: Add a secure password reset process using time-limited tokens sent to the user’s email. For example, use `secrets.token_urlsafe(32)` and set a 15-minute expiration.

Conclusion

The penetration test of the Secure Document Sharing System showed strong security across all tested areas. The application correctly implements the required security features from the CS 419 project, this includes bcrypt password hashing, role-based access control (RBAC), input validation, Fernet encryption, secure session handling, and proper security headers.

No critical or high-severity issues were found. The two low-severity findings (self-signed certificate and missing password reset) were found in the development setting and do not pose real risks in the current setup or future if we were to deploy.

The medium-severity issue with debug mode should be fixed before any production use by setting `debug=False` and using environment variables for configuration. Overall, through scripting and using external tools we were able to get a good understanding of where our project was in terms of security and foundation while also following the instructions.